

Messaging in Rhapsody

A message is the basic unit of data processed by the Rhapsody Engine. Messages can contain arbitrary binary or text data but frequently have a structure and content that conforms to a messaging standard.

Message Standards

Rhapsody provides support for a wide range of health messaging standards. The most common is [HL7 V2\(opens in a new tab\)](#).

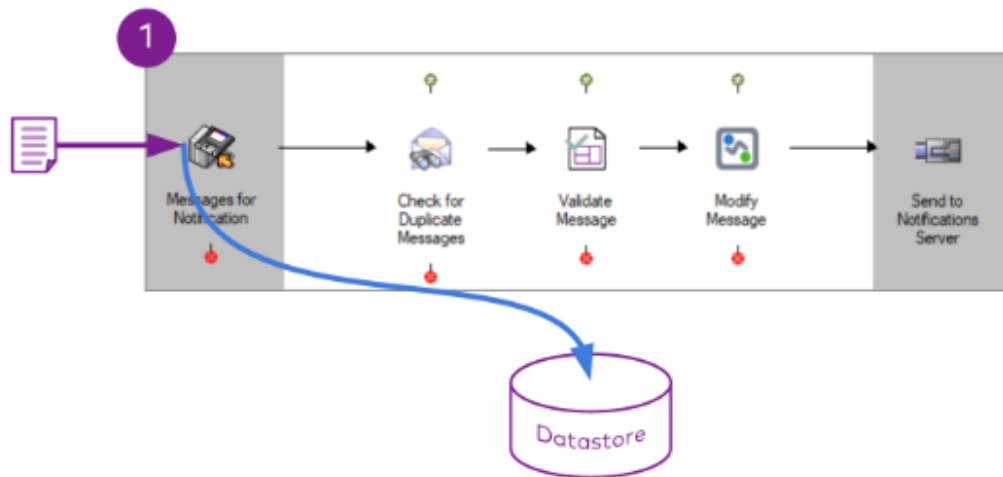
- Message standards are loaded through the Library Wizard in the EDI Message Designer where you can:
 - Create definitions for most common messaging standards with little effort.
 - Modify or create messaging standards (such as [CSV\(opens in a new tab\)](#) format).
- XML-based message standards can be loaded directly into Rhapsody using an XSD ([XML Schema Definition\(opens in a new tab\)](#)).
- Messages can be changed during processing. These changes can be relatively minor, where an item of information is inserted into an existing field. Or they can completely alter the structure of the message.

The Message Processing Path

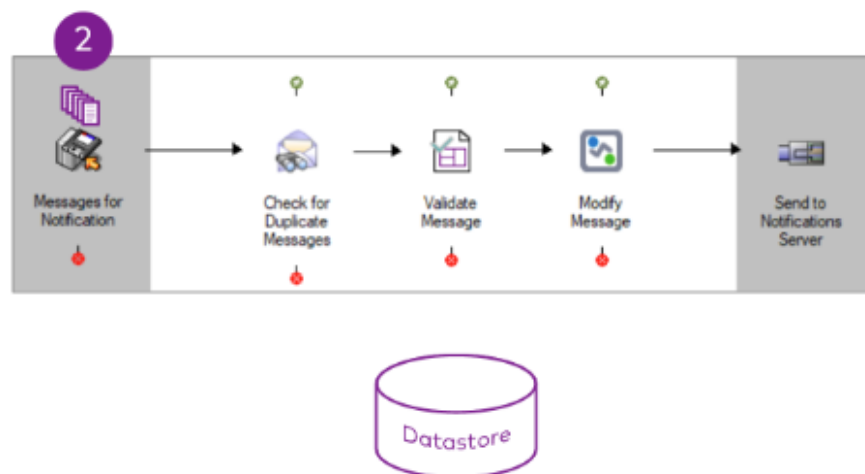
The Rhapsody processing model uses message queues and transaction records to ensure messages are saved at key points in the route as they are processed.

The Message Processing Path

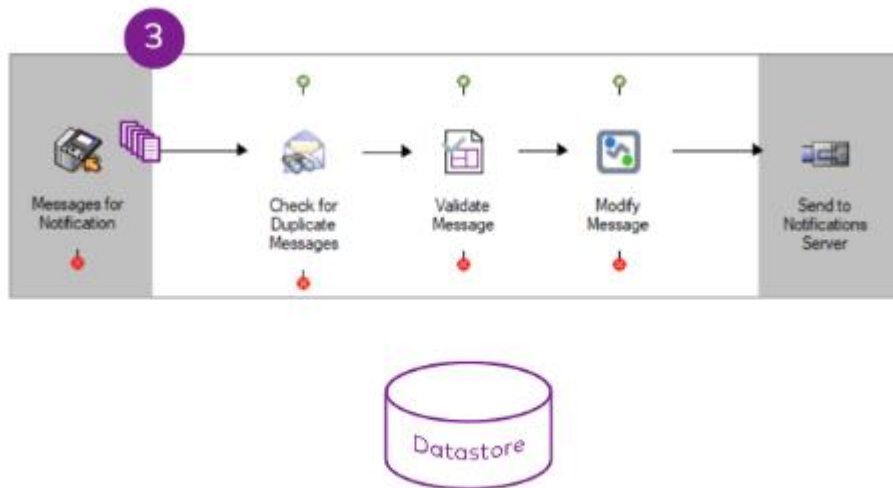
The Rhapsody processing model uses message queues and transaction records to ensure messages are saved at key points in the route as they are processed.



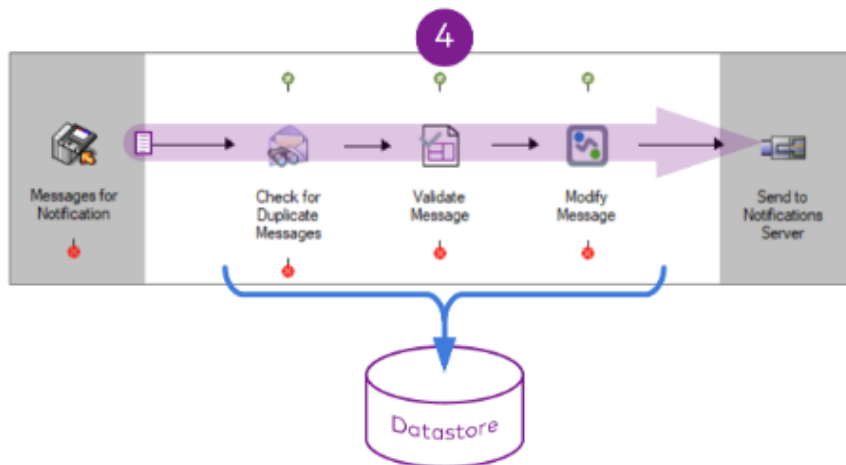
The message is received by the input communication point and is immediately saved to the **Datastore**.



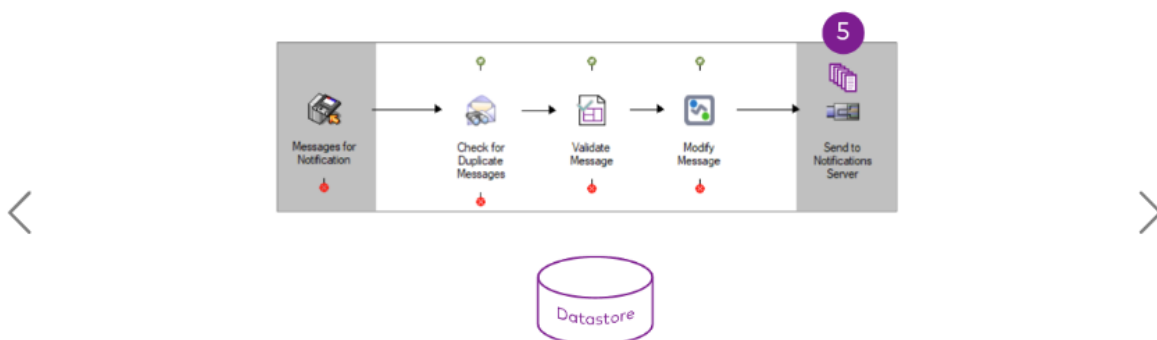
The message is placed onto the communication point input queue.



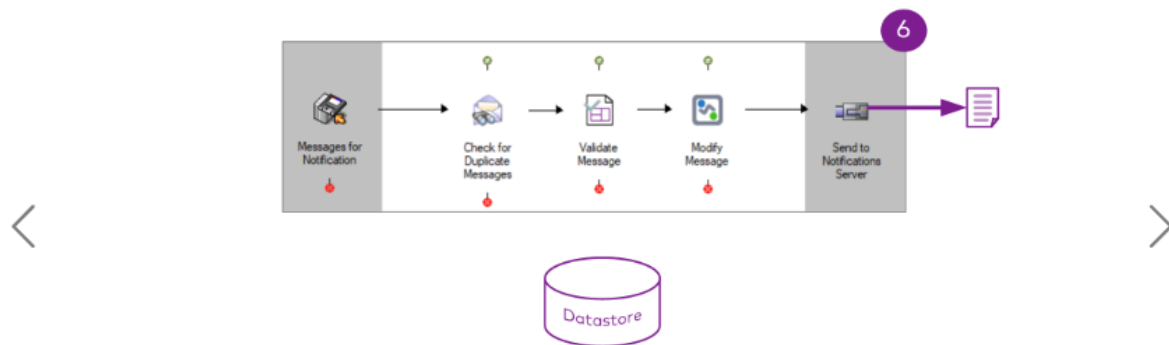
The message is passed to the route processing queue.



The message is processed through the route filters. Multiple messages can be processed by the route at the same time. If a filter changes the message in any way, a copy of the message as it leaves the filter will be saved to the datastore.



The message is passed to the output communication point queue.



6. The message is sent from the **output communication point** to an external receiving system.



If there are several filters on a route, each of which changes the message, a copy of the message is saved each time the message exits a filter. This can increase the processing load on the engine. As a result, we recommend that routes are designed with the minimum number of filters needed to fulfill their requirements.

An exception is the No-operation filter, which does not alter the structure or content of the message. Filters of this type are often used as branching points in a route or to visually document message flow at different stages of the route.

You can view copies of all messages saved to the datastore in the Management Console by selecting the route and choosing to view its archived messages.

System Failure Protection

At each processing stage, a transaction record protects the message from the effects of a system failure. If an engine restart occurs, incomplete transactions are rolled back and restarted as soon as processing resumes.

Limitations

While Rhapsody can handle messages of any size, there are practical limits that are dictated by performance and hardware requirements. Messages of a few kilobytes up to tens of megabytes in size typically will not present a problem to the processing engine.

Message Properties

When Rhapsody processes a message, it is frequently useful or necessary to associate additional metadata with it. This metadata is separate from the content or body of the message and can include:

- Information identifying the message's type
- Date and time of its arrival

- Name of its originating application or facility

Metadata associated with a message is referred to as Message Properties. Changes in a property value are written to the Rhapsody datastore in the same way as changes made to the message body.

Setting Message Properties

Message properties are either implicitly or explicitly defined. Both implicitly and explicitly defined message properties are displayed at each processing step in the Management Console on the **Message View** page.

Implicitly defined properties

These are properties set on the message by the engine. The following properties, for example, are defined for all messages on arrival:

- **InputTime**
A timestamp identifying when the message was received by the route. This property is expressed as the number of milliseconds since 1 January, 1970 (the Unix / Java Epoch).
- **rhapsody:InputCommunicationPoint**
The name of the communication point on which this message was received.
- **rhapsody:InputCommunicationPointId**
The internal (Rhapsody-generated) ID of the communication point on which this message was received.

These properties are set when a message is parsed by a message definition:

- **MessageName**
The current message type.
- **rhapsody:InputMessageType**
The input message type; for example, ADTA01.
- **rhapsody:OutputMessageType**
The output message type (this is typically the same as the **MessageName** property); for example, ACK.

Explicitly defined properties

- Properties set by the Encoding System used by the message body.
- Properties set on the message by filters as it passes through the route. The Property Population filter, for example, can extract the name of the message's **SendingFacility** field from its content and set this value as a message property.

Both implicitly and explicitly defined message properties are displayed at each processing step in the Management Console on the **Message View** page.

Interpreting Messages

By default, Rhapsody does not attempt to interpret the structure of a message. Typically, these messages simply pass through the engine unaltered. It is sometimes possible, however, to locate specific message content and use that information to direct the path it takes through the engine.

Interpreted Messages

The content of interpreted messages conforms to a well-known structure, for example: HL7, CSV, XML, or JSON. A message's structure is interpreted by the engine using a message definition specified during configuration. This allows the message's fields or components to be identified and their values extracted or updated.

Each field in an interpreted message is addressable, which means that it can be referred to by its logical path within the message structure. An XPath-like syntax (for example, node/element/.../field_name) is used. The precise path details are dependent on the message's structure.

Messages handled by Rhapsody fall under four main structures/standards:

Fixed Width Message Structures

- Typically have a rigid structure where the message contains a single line, and data fields are defined by their offset from the start of the message.
- Contain no intrinsic information about structure and therefore require external information to formalize the definition.
- Might be batched (containing multiple lines), requiring identification and possibly extraction to individual messages prior to processing.

Ad-hoc Message Structures

- Are defined locally and by agreement between the sending and receiving parties.
- The structure is typically defined by including known field delimiters within the message.
- A frequently used example is the CSV format as exported by the Microsoft product suite. The CSV format is a delimited format using commas to separate the fields. However, the term is often used to describe other formats, such as tab-delimited.
- Ad-hoc messages are typically single line messages but may be received in batches (multiple lines per batch, each line being a message).

IT Industry Standards

- Message structures are typically defined by a relevant industry consortium or the International Standards Organization (ISO) and are well documented.
- The most common example is XML formatted messages. Rhapsody supports implicit processing of XML messages based on XPath field descriptions, but complex structures are preferably managed by using an XSD (XML Schema Definition).
- Fields (elements) in an XML message are defined in a tree hierarchy with well-defined parent/child and sibling relationships.

There are numerous sector-specific standards defined for more specific purposes rather than for general data interchange. For example, messaging using the ASTM (American Society for Testing and Materials) standard (or a variant) is typically encountered when working with instrumentation in health environments.

Health Industry Standards

The most common example of a Health Industry messaging standard is HL7 V2, which is what we will refer to mostly in this course. But there are others such as HL7 V3, HL7 FHIR (Fast Healthcare Interoperability Resources) and CDA (Clinical Document Architecture).

HL7 Messages

Throughout this course we will make frequent reference to the HL7 V2 ([opens in a new tab](#)) specification. We typically refer to this as HL7. You should already be familiar with the HL7 message standard. If not, expand the heading below for a quick overview.

Interpreted Message Examples

The following examples show message types commonly interpreted by Rhapsody.

HL7

An HL7 V2.4 message using standard field delimiters, including a | (pipe) to separate fields, ^ (caret) to separate composites (sub-fields), and a ~ (tilde) to separate repeats of a field.

CSV

A file containing comma-separated values. As the name suggests, these files typically use commas to separate fields within each record. A line break is used to separate records within one file.

XML

An XML message containing a FHIR patient resource.

Character Encoding, and Single vs. Batch Messaging

Character Encoding

Character encoding determines how the text in a message will be processed and presented. If a message is processed using an incorrect character encoding, text in the message can become corrupted. In some cases, incorrect character encoding can cause a message to become invalid and it won't be processed. Even in environments where English is the predominant language, special characters can find their way into a message from text that has been copied and pasted into a report or from an automated system.

The same content saved with different character encodings will use different bytes when saved to disk or sent to another system. Consider the following text that could be included in a message as part of a laboratory test result:

Test performed @ 23°C

The degree symbol in this text would be encoded in the following coding systems as:

ISO8859-1	UTF-8	UTF-16
b0	c2 b0	00 b0

In many character encoding schemes, non-accented Latin characters are encoded the same. However, most special characters (for example, the degree symbol - °), accented characters (for example, é), and non-Latin characters (for example, Αθήνα or 東京) will differ.

Rhapsody maintains a reference to the message's character encoding using Java encoding definitions. This information is used to decode the message when the text in the message needs to be processed. If a message encoding is not explicitly set, Rhapsody will attempt to determine it using either heuristics or a system setting when the message is processed.

The character encoding for a message can be set when it arrives at an input communication point. A character encoding can be changed during processing using the Character Encoding Translation filter.

Single vs Batched Messages

While messages received by Rhapsody generally contain a single or discrete message, it is not uncommon that messages are supplied as a batch (that is, several discrete messages grouped together into a batched message).

Common examples of batched messages include:

- CSV messages where each line of the message needs to be processed separately.
- HL7 messages where the message conforms to the [HL7 batch specification\(opens in a new tab\)](#).

Message debatching

Rhapsody provides two options for splitting batched messages into single messages for processing.

- The Directory communication point contains options for splitting files into multiple messages as they are read from disk.

- The Batch / De-batch filter can split messages during processing in Rhapsody.

Message batching

When Rhapsody processes a set of messages (either received individually or split up as a result of de-batching), they can be grouped together and assembled into a single message.

Examples of batching and de-batching are described in a later module of this course.

Exercise - Message Structures

Overview

This exercise uses EDI Explorer to examine the structure and content of HL7 and CSV messages.

Resources

Before beginning this exercise, you should have already downloaded the *Associate Files and Definitions.zip* file from the "Resources" module at the start of this course and extracted the files to a location on your machine.

Two test messages and their definition files are required to complete this exercise. These files can be found in the *.\Associate Files and Definitions\102 - Message Structures* folder in the *Associate Files and Definitions.zip* file.

You will use the following files:

- ***Patients.csv***
A test CSV message.
- ***Academy-CSV-WardList.s3d***
A Rhapsody Messaging Toolkit CSV Message Definition file.
- ***Academy_HL7v2.4_testMsg.edi***
A test HL7 message.
- ***Academy-HL7v2.4-ADT.s3d***
An HL7 version 2.4 Rhapsody Messaging Toolkit Message Definition file.

For this and later Associate course exercises, you can ignore the two .pcf files in the *Test Messages* directory. These are Parser Control Files and are compiled versions of the message definitions found in the same directory. They are generated automatically by Rhapsody.

Exploring an HL7 File

HL7 messages are composed of segments whose number, order and composition are dependent on the message type. Segment identifiers are three-character codes located at the start of the segment, for example:

- MSH (Message Header)
- EVN (Event)
- PID (Patient Identification)

Each segment is composed of multiple fields in a defined order.

Tasks

This exercise features an HL7 message containing randomly generated data. It conforms to the HL7 V2.4 standard and uses the pipe character (|) to separate fields and a carriage return to separate segments.

To load the test message into EDI Explorer:

1. Open EDI Explorer.
From the Windows Start menu, select **Rhapsody 7 > EDI Explorer**.
2. Load the message definition.
 1. From the **File** menu, select **Open Definition**, then navigate to the directory containing the resources for this exercise on your machine.
 2. Select and load the *Academy-HL7v2.4-ADT.s3d* message definition.
3. Load the test message.
From the **File** menu, select **Import Flatwire Message From File** and navigate to the directory containing the resources for this exercise.
4. Select the **HL7v2.4_testMsg.edi** file and load the test message.
You can also use **File > Open** to open the test message, but it will not be listed by default. If you use this method, change the **File Type** option to **All Files** to include *.edi* files in the list.

Test message loaded into EDI explorer.

In EDI Explorer, you should see the following details of the message:

- Message type
ADTA01
- Message name
Academy_HL7v2.4_testMsg.edi
- Parsed message
Structure and content of the message after parsing by the message definition
- Flat wire message
Message content as it would be seen in a text editor such as Windows Notepad.
Messages in this format are known as 'flat wire' messages.
- Validation errors and warning messages
Errors resulting from the message's parsing by the message definition. The test message will not generate any errors.
- Loaded message definition
Name of the message definition used to parse the test message: *Academy-HL7v2.4-ADT.s3d*.

Inspect a message field

1. Select the + sign next to the MSH segment to expand the segment.
2. Expand the SendingFacility field to reveal its subfields.
3. Select the NamespaceID field. The content of this field is contained within the " " characters.

The content of a field selected in the top panel is highlighted in the middle flat wire message panel. In this example, the content of the MSH.Sending Facility.NamespaceID field is **CANNT**.

Inspecting the test message in EDI explorer.

In the HL7 sample message, locate the MessageType field in the MSH segment and expand the field values. What is the value of the MessageType subfield? Write your answer in three characters (for example, PR1).

SUBMIT

Examine the HL7 sample message in EDI Explorer. Locate the MessageType field in the MSH segment and expand the field value. What is the value in the TriggerEvent subfield? Write your answer as three characters (for example, PR1).

SUBMIT

What is the value of the subfield Surname found in PID.PatientName[0].FamilyName?

SUBMIT

What are the two segment types present in Procedures.Procedures?

Write your three-character answers separated by a space (for example, PID EVN).

SUBMIT

Exploring a CSV file

The CSV format is used to export and transfer data between spreadsheets and can be used by other software as a data transfer standard. It has no structure or rules, except for defining the character used to separate fields. Typically, this is a comma (,) but could also be the tab or pipe (|) characters.

The CSV format allows multiple, unrelated records per file, with the first record often containing the data field names. The following example contains hospital ward details for a set of fictitious patients.

Tasks

1. Open EDI Explorer.
2. From the **File** menu, select **Open Definition**. Locate and select the **Academy-CSV-WardList.s3d** message definition.
3. Confirm its name appears in the status bar at the bottom of the window.

EDI Explorer can only use one definition at a time. This means that any previous message definitions will be replaced when this file is loaded.

4. From the **File** menu, select **Open**. Locate and select the **Patients.csv** file.

You may need to change the File Type to **All Files** to see the CSV test file included in the list.

5. Expand the **Patient** segment for the message.

CSV message in EDI Explorer.

The CSV test message has a single segment (**Patient**) which is repeated multiple times. Note that the repeat count starts at 0 (zero).

The characters displayed within square brackets [] at the end of each line in the **Loaded Message** panel are the hexadecimal representations of the segment terminator characters. In the test message included with this exercise, this will be **[0D][0A]** - **Carriage Return** and **Linefeed**.

Examine the CSV sample message in EDI Explorer. What is the value of the MiddleName field for the patient with a PatientId of 87619-9902?

Write your answer exactly as it appears in the data.

SUBMIT

Including the header row, how many records are there in the CSV file?

Type your answer as a numeric value.

SUBMIT

Exercise - Message Processing

Overview

This exercise examines messages as they pass along a route pipeline. Three examples will be used:

1. **Single Message processing**

Examines the path of a single message through Rhapsody.

2. **Batched Message processing**

Examines the path of a batched CSV message as it is separated into its component records during processing. This example illustrates how changes to message properties are tracked as the message moves through the route.

This exercise also highlights the core concepts used by Rhapsody when handling messages:

- **Persistence**

The engine guarantees message delivery according to business logic by saving each message to the Rhapsody datastore. This occurs as a message is received by Rhapsody, exits Rhapsody, and whenever its content or property values are modified by the action of a Rhapsody component.

- **Parsing**

The message's structure is validated against a definition file. This allows the message to be treated as 'intelligent', making its fields and properties addressable.

Resources

This exercise uses an RLC available in the *.\Associate Files and Definitions\102 – Working with Messages* folder in the resources Zip file available at the start of the course.

Setup

Start by importing the *Message Processing.rlc* file into your Rhapsody engine using the following steps.

1. From the Windows Start menu, open Rhapsody IDE and log into your local Rhapsody server.
2. From the **File** menu, select **Open** to open the Reload Configuration Wizard.
3. Select **Browse** to navigate to the location of the exercise RLC file. Select the RLC file, then select **Next**.
4. Select **Start All** as the Starting Options for the routes and communication points.
5. Select **Finish** to load the entire contents of the RLC file into Rhapsody.
6. If a question is displayed asking whether passwords should be imported along with the configuration, please choose to **Remove Passwords**.
7. Enter a check-in comment (not required), then select **Load**.

You can now view the Exercise routes and communication points in the IDE Workspace. Saving and reloading an RLC file will be covered in more detail in the *Working with the IDE* module.

Single Message Route

This example uses the **Single Message** route located in the *Rhapsody Academy/Rhapsody Associate/REN-102/Message Processing/1. Single Message Route* folder of the IDE Workspace. All three components in the folder must be running, as indicated by the ► icon alongside their names.

The **Single Message** route generates a new test message every five seconds then sends it into the route for processing. We will use the Management Console to view the changes to the message at each stage of its progress through the engine.

Tasks

Use the image below the listed steps for guidance.

1. 1

Log in to the Management Console.

Go to [https://localhost:8444/rhapsody/login\(opens in a new tab\)](https://localhost:8444/rhapsody/login(opens in a new tab)), or select the Management Console icon in the IDE Toolbar, to log into the Management Console. Your username and password are the same as you use to log into Rhapsody IDE.

2. 2

From the **Monitoring** menu, select the **Routes** link to list the currently configured routes, along with their status and message processing counts.

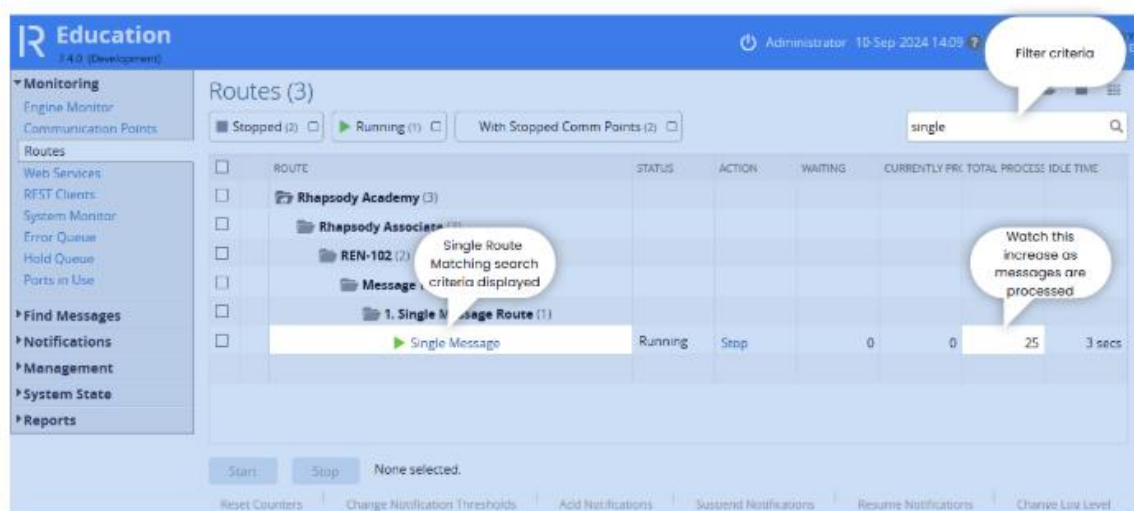
3. 3

Type *single* into the filter box in the top-right corner of the screen.

Only those folders and routes containing *single* in their title will be listed. Search criteria in Management Console are not case-sensitive.

4. 4

Ensure the **Single Message** route is started and actively processing messages by looking at the **Total Processed** count. Refresh the screen and watch the value increment.



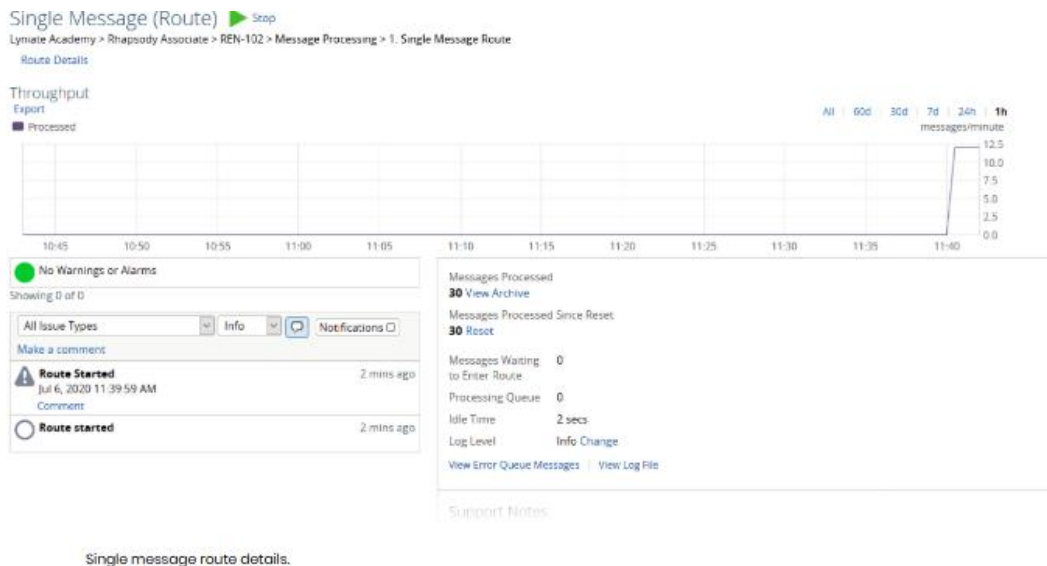
Single message route

Route Information

1. 5

Select the **Single Message** route to open the **Route Summary** window.

This window includes a **Throughput** graph showing variations in message processing over time along with a panel giving route statistics, such as the total number of messages processed and the number of messages on input or processing queues.

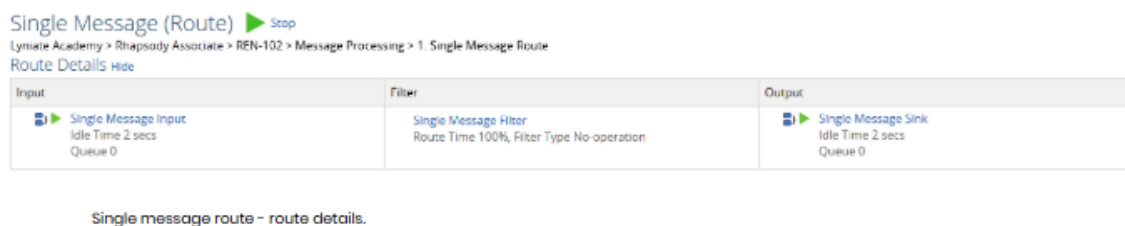


Route Details

1. 6

Select the **Route Details** link to show the names of its associated communication points and filters. This panel shows:

- The route's input and output communication points, along with their activity statistics.
- The name and type of all filters associated with the route.



Message Archive

1. 7

Select the **View Archive** link in the **Messages Processed** panel to list the messages that have passed through this route, sorted by age.

Education

7.4.0 (Development)

Administrator

10-Sep-2024 14:09

INTEGRATION ENGINE

Monitoring

Engine Monitor

Communication Points

Routes

Web Services

REST Clients

System Monitor

Error Queue

Hold Queue

Ports in Use

Find Messages

Input Search

Output Search

Route Search

Message Collector Search

Saved Searches

Recent Results

Notifications

Management

System State

Reports

Single Message Archive

Completed Reload

No saved searches

Timeframe

From / To

Show Last

From

To

Search Order

Search Options

Hide

Routes

Single Message

Received By

Any Component

Message Contains

Case-Sensitive

Ignore Case

Regular Expression

Search

Reset

SEARCH CRITERIA APPLIED

Quick Filter

TIME	ROUTE	RECEIVED BY	NEXT DESTINATION	
☐ Jul 6, 2020 11:43:05.285	Single Message	Single Message Input	Single Message Filter	
☐ Jul 6, 2020 11:43:00.283	Single Message	Single Message Input	Single Message Filter	
☐ Jul 6, 2020 11:42:55.260	Single Message	Single Message Input	Single Message Filter	
☐ Jul 6, 2020 11:42:50.253	Single Message	Single Message Input	Single Message Filter	
☐ Jul 6, 2020 11:42:45.250	Single Message	Single Message Input	Single Message Filter	
☐ Jul 6, 2020 11:42:40.245	Single Message	Single Message Input	Single Message Filter	
☐ Jul 6, 2020 11:42:35.244	Single Message	Single Message Input	Single Message Filter	
☐ Jul 6, 2020 11:42:30.240	Single Message	Single Message Input	Single Message Filter	

38 message(s) found

Reject

Redirect Copy

Download

Download All

Single message route archive.

Message View

1. 8

Select the first message to display the **Message View** window.

This window includes the **Message Events**, **Message Properties** (showing the message's metadata) and **Message Body** (the message's content, displayed in Tree or Flat Wire format) panels.

Message View 1.922496.7207

Displaying message 1 of 38 from the result set named 'Single Message Archive' [Back to Result Set](#)

Reinject [Redirect Edited Copy](#)

Message

Communication points and filters responsible for processing message

Always Show [Last refreshed at 11:43](#)

COMPONENT	TIMESTAMP	DESCRIPTION
Single Message Input	Jul 6, 2020 11:43:05.284	Received by communication point
Single Message	Jul 6, 2020 11:43:05.285	Placed on route
Single Message Filter	Jul 6, 2020 11:43:05.285	Filtering succeeded; took 0.003 ms
Single Message Sink	Jul 6, 2020 11:43:05.319	Delivered to communication point
	Jul 6, 2020 11:43:05.319	Sent by communication point, total proc...

Message Properties

Property Name	Property Value
InputTime	1593992585284
rhapsody:InputCommunicationPoint	Single Message Input
rhapsody:InputCommunicationPointId	22

Showing 3 of 3

Message Body

Tree View [Flat Wire](#)

Show Whitespace [Encoding: US-ASCII](#) Size: 258 bytes

Content of message body

```
PatientId,GivenName,MiddleName,FamilyName,Sex,Ward,Room,Bed\r\n
84568-4564,John,Quentin,Cardinal,M,9,1,1\r\n
84568-7664,Bert,Patrick,Pudding,M,9,1,4\r\n
87619-9992,Lila,Betty,Johnson,F,9,2,2\r\n
84568-4341,Max,Philip,Headroom,M,9,3,1\r\n
84568-3168,Theresa,,Green,F,9,2,3\r\n
```

Download

Single message route - message view.

1. Select the **Single Message Input** event, then select **Next Message Event** to move through the **Message Events** tree until **Single Message Sink** is reached. Note the following features:

- The **Message Property** values are associated with the message by the input communication point and do not change as the message moves through the route.
- The **Message Body** (content) does not change as the message moves through the route.
- The **Processing Time** for each event is shown in the **Description** column.
Note that the filter processing time is in milliseconds while the time on

the route is in seconds. You can adjust the width of this column to bring all the details into view.

Processing Time

Processing is quick in the **Single Message** route because the message is small and the current load on the Rhapsody engine is light. In the above example, the total processing time is 0.002 seconds. This is the time from when the message was accepted into the route's input queue to when it left its output queue.

This time includes the processing time on each of the route's communication points and filters as well as:

- The time spent on queues waiting for filters to become available.
- The time taken to apply changes to the message's content and/or properties.
- The time taken to write a copy of input and output messages to the Rhapsody archive.

Batched Message Route

This example uses the **CSV** messages route located in the *Rhapsody Academy/Rhapsody Associate/REN-102/Message Processing/2. Batched Message Route* folder in the IDE Workspace. All three components in the folder must be running, as indicated by the ► icon alongside their names.

The **CSV messages** route uses the same test message as in the previous example but treats it as a batched message that can be separated into individual files. Message properties are associated with each de-batched message for viewing in the Management Console.

Tasks

- 1.

Log into the Rhapsody **Management Console**.

- 2.

From the **Monitoring** menu, select the **Routes** link to view the currently configured routes.

Confirm that the **CSV Messages** route is present and actively processing messages.

- 3.

Select the **CSV Messages** route to open the **Information** screen.

- 4.

Select the **View Archive** link to view the messages processed by this route.

Message view - at the input communication point

1. 5

Select the most recently processed message to open the **Message View** screen.

2. 6

Ensure the first event, **Batched Message Input**, is selected to show the state of messages arriving at the route's input communication point.

Message View 1.984192.7689

Displaying message 1 of 1 from the result set named 'CSV messages Archive' [Back to Result Set](#)

Reinject Redirect Copy Redirect Edited Copy

Message Events

Always Show Outputs

Last refreshed at 17:00

COMPONENT	TIME	DESCRIPTION
Batched Message Input	Jul 6, 2020 17:00:45.644	Received by communication point
CSV message sink	Jul 6, 2020 17:00:45.646	Placed on route
Message De-batch	Jul 6, 2020 17:00:45.657	Filtering succeeded; took 11.469 ms
De-batched Message Sink		
De-batched Message Sink		
De-batched Message Sink		
De-batched Message Sink		
De-batched Message Sink		
De-batched Message Sink		

Message Properties

Property Name	Property Value
InputTime	1594011645644
rhapsody:InputCommunicationPoint	Batched Message Input
rhapsody:InputCommunicationPointId	16

Showing 3 of 3

Message Body

Free View Flat Wire

Show Whitespace Encoding: US-ASCII

Size: 258 bytes

```
PatientId,GivenName,MiddleName,FamilyName,Sex,Ward,Room,Bed\r\n84568-4564,John,Quentin,Cardinal,M,9,1,1\r\n84502-7664,Bert,Patrick,Pudding,M,9,1,4\r\n87619-9982,Lila,Betty,Johnson,F,9,2,2\r\n84568-4341,Max,Philip,Headroom,M,9,3,1\r\n84568-3168,Theresa,,Green,F,9,2,3\r\n
```

Download

Note that there are three message properties associated with the message at this point in the route. Additionally, the full batched message content is shown in the **Message Body** panel.

Message View - at the Output communication point

1. 7

Select the first of the **De-batched Message Sink** events to show the state of messages leaving the route's output communication point.

The screenshot displays the 'Message View 1.984448.7702' interface. At the top, it indicates 'Displaying message 1 of 1 from the result set named 'CSV messages Archive'' with a 'Back to Result Set' link. Below this are buttons for 'Reinject', 'Redirect Copy', and 'Redirect Edited Copy'. The 'Message Events' section is expanded, showing a list of events. A callout bubble points to the first 'De-batched Message Sink' event, stating 'First destination event selected'. The events table has columns for 'COMPONENT', 'Timestamp', and 'DESCRIPTION'. The 'Message Properties' section is also expanded, showing a table of properties. A callout bubble points to the 'BatchIndex' property, stating 'Two properties have been added to message'. The 'Message Body' section is expanded, showing a 'Tree View' and a 'Flat Wire' view. A callout bubble points to the 'Flat Wire' view, stating 'Message body contains first de-batched row'. The 'Flat Wire' view shows the message content: 'PatientId,GivenName,MiddleName,FamilyName,Sex,Ward,Room,Bed\r'. The 'Size: 60 bytes' is indicated on the right. A 'Download' button is at the bottom left.

COMPONENT	Timestamp	DESCRIPTION
Batched Message Input		Received by communication point
CSV messages	Jul 6, 2020 17:00:45.646	Placed on route
Message De-batch	Jul 6, 2020 17:00:45.657	Filtering succeeded; took 11.469 ms
De-batched Message Sink	Jul 6, 2020 17:00:45.661	Delivered to communication point
De-batched Message Sink	Jul 6, 2020 17:00:45.663	Sent by communication point, total proc...
De-batched Message Sink		
De-batched Message Sink		
De-batched Message Sink		
De-batched Message Sink		
De-batched Message Sink		

Property Name	Property Value
BatchIndex	0
InputTime	1594011645644
rhapsody:InputCommunicationPoint	Batched Message Input
rhapsody:InputCommunicationPointId	16
TotalNumberOfBatchedMessages	6

Showing 5 of 5

Message Body

Tree View

Flat Wire

Show Whitespace Encoding: US-ASCII Size: 60 bytes

PatientId,GivenName,MiddleName,FamilyName,Sex,Ward,Room,Bed\r

Download

Batched message at the output communication point

Compare the input and output message views and note the following differences:

- The de-batching process produces six individual messages, each with its own **De-batched Message Sink** event.
- Selecting each of the **Batched Message Sink** events reveals that each output message has an incrementing BatchIndex property value (zero-based)
- The **Message Body** corresponding with each event lists a single record, with the header labels (PatientId, GivenName, etc.) present in the first message.

The message is de-batched using the line terminator \n identified on the **Configuration** tab of the **Batch/De-batch** filter's **Properties** dialog. This must match the terminator in the original message.

The **InputTime** displayed on the **Message View** page as a **Message Property** is a long number - what does it represent?

Message Properties

Property Name	Property Value
InputTime	1562529561208
rhapsody:inputCommunicationPoint	Single Message Input
rhapsody:inputCommunicationPointId	124

- ☐ The number of milliseconds since January 1, 1970
- ☐ The time taken by the message to move through the engine (in microseconds).
- ☐ The number of seconds since January 1, 1900.
- ☐ The number of milliseconds since the engine was last started.

SUBMIT